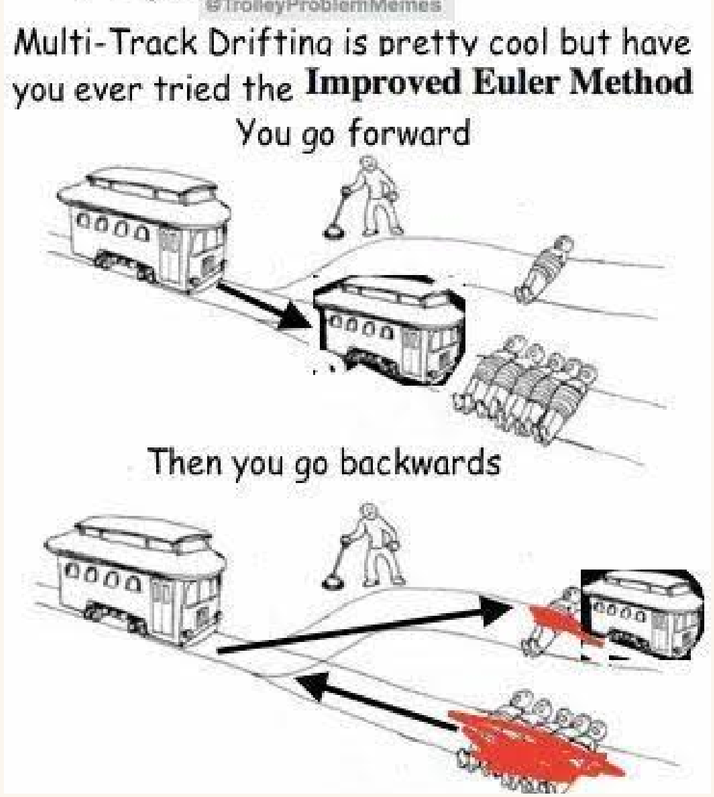
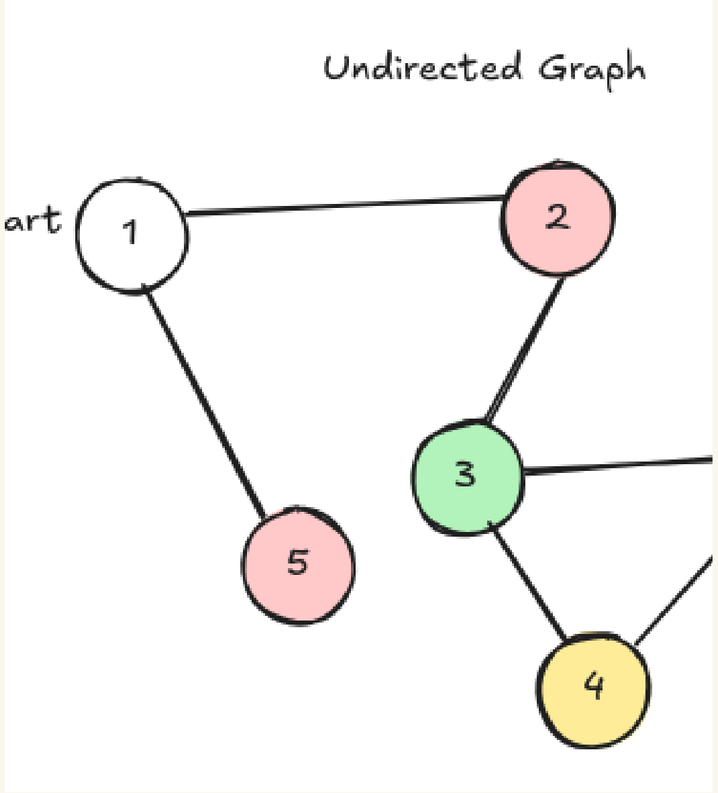
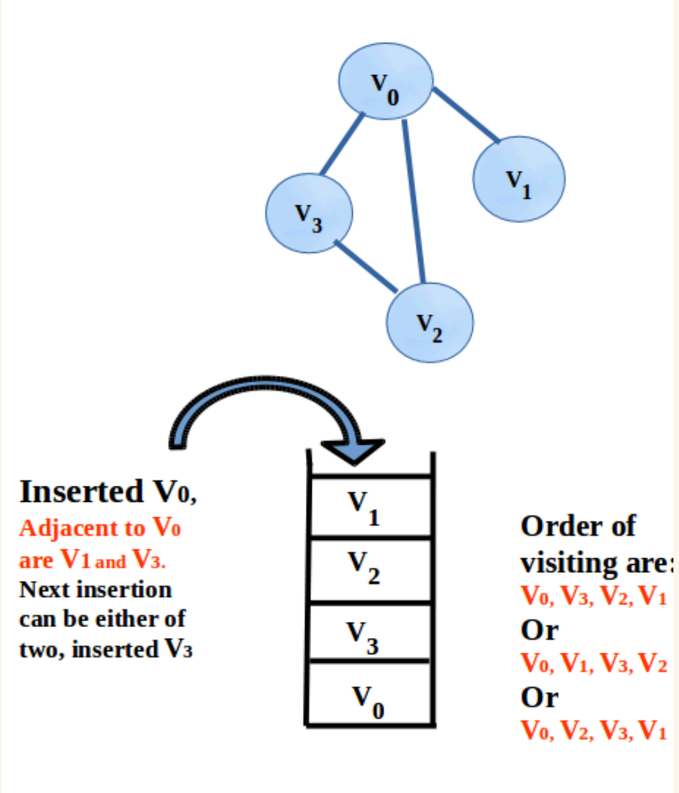
Parallel Breadth-First Search on the Orkut Social Network Using CUDA

6710405427 พีรณัฐ เมฆวิศษฏ์

CS343-65 (69-1) Parallel Computing with CUDA





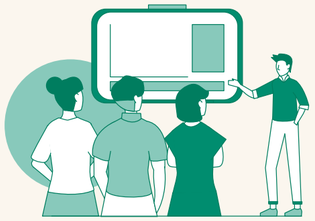


Problem

Graph Traversal
the process of visiting (checking and/or updating) each vertex in a graph. Such traversals are classified by the order in which the vertices are visited. Tree traversal is a special case of graph traversal.

wiki/Graph_traversal

Dataset - Orkut social network



Dataset origin

Orkut is a free on-line social network where users form friendship each other.

Stanford Network Analysis Platform (SNAP) is a general purpose network analysis and graph mining library.

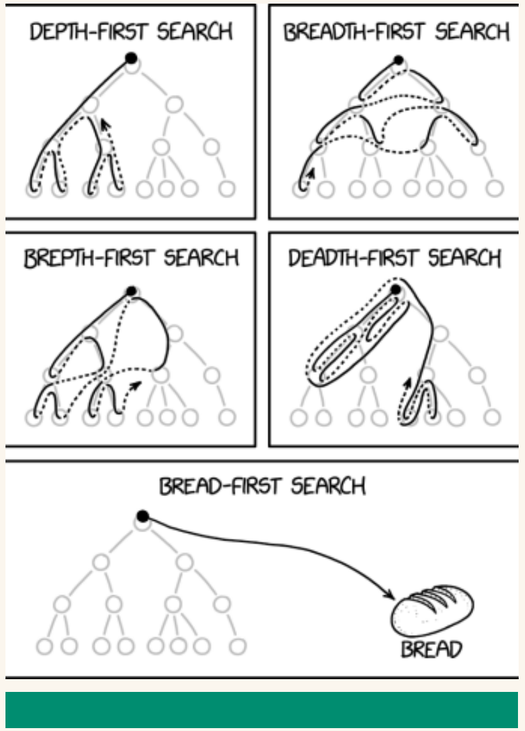


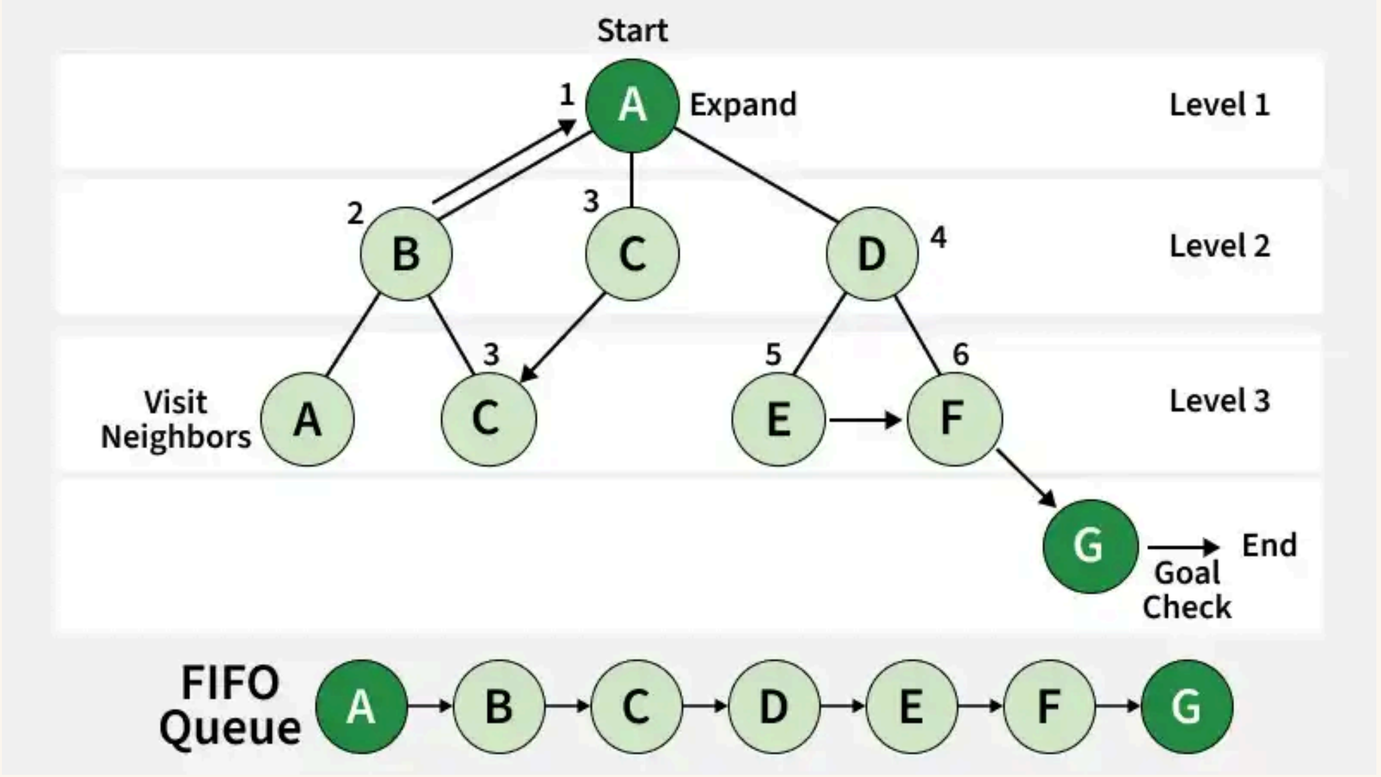
Dataset information

Undirected Orkut network
Nodes : 3,072,441
Edges : 117,185,083
Adjacency Entries : 234,370,166

Dataset statistics	
Nodes	3072441
Edges	117185083
Nodes in largest WCC	3072441 (1.000)
Edges in largest WCC	117185083 (1.000)
Nodes in largest SCC	3072441 (1.000)
Edges in largest SCC	117185083 (1.000)
Average clustering coefficient	0.1666
Number of triangles	627584181
Fraction of closed triangles	0.01414
Diameter (longest shortest path)	9
90-percentile effective diameter	4.8

This data is provided by Alan Mislove et al.





Breadth First Search

an algorithm for searching a tree data structure for a node that satisfies a given property. It starts at the tree root and explores all nodes at the present depth prior to moving on to the nodes at the next depth level. Extra memory, usually a queue, is needed to keep track of the child nodes that were encountered but not yet explored.

wiki/Breadth-first_search

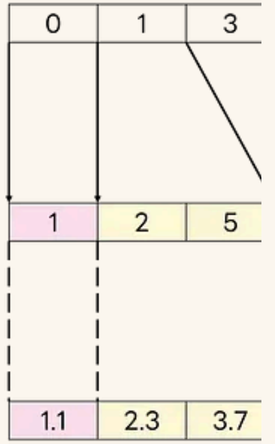
Compressed Sparse Row (CSR)

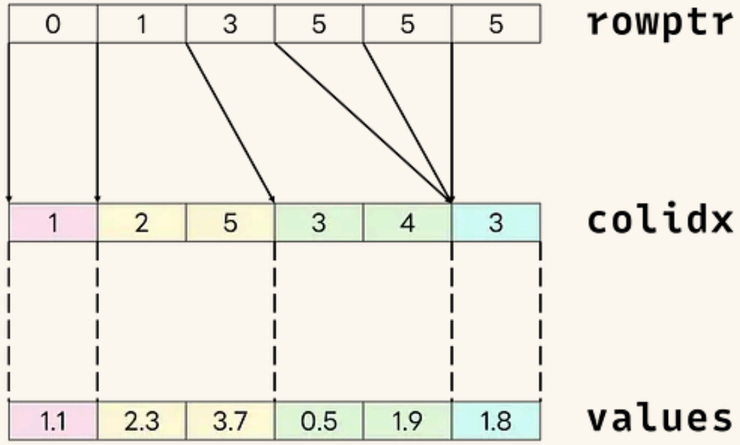
เป็นรูปแบบยอดนิยมสำหรับเก็บข้อมูลกราฟ (Graph) โดยเฉพาะกราฟที่มีความบาง (Sparse Graph หรือกราฟที่มีเส้นเชื่อม/Edges น้อยเมื่อเทียบกับจำนวนจุดยอด/Vertices ทั้งหมด)

Row Pointers (offsets) = บอกตำแหน่งเริ่มต้นและสิ้นสุดของ Neighbor
Column Indices (neighbors) = เก็บหมายเลข Vertex ปลายทาง

	0	1	2	3	4	5
0	0	1.1	0	0	0	0
1	0	0	2.3	0	0	3.7
2	0	0	0	0.5	1.9	0
3	0	0	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	1.8	0	0

Sparse matrix





best possible sequential solution.

1. กำหนด source vertex และกำหนด distance[source] = 0
2. นำ source ใส่ใน Queue
3. นำ vertex ออกจาก Queue ทีละตัว
4. อ่าน neighbors จาก CSR โดยใช้ offsets[u] ถึง offsets[u+1]
5. ถ้า neighbor ยังไม่เคยถูกค้นพบ ให้กำหนดระยะทางและใส่ลง Queue
6. ทำซ้ำจน Queue ว่าง

```cpp
BFSResult runSequentialBFS(const Graph& graph, int source ) {
    BFSResult result;
    result.source = source;

    result.distance.assign(graph.numNodes, -1);

    std::queue<int> frontier;


    // root vertex
    result.distance[source] = 0;
    frontier.push(source);

    while (!frontier.empty()) {

        int u = frontier.front();
        frontier.pop();

        result.reachedVertices++;

        int currentDistance = result.distance[u];

        if (currentDistance > result.maxDistance) {
            result.maxDistance = currentDistance;
        }

        for (int i = graph.offsets[u]; i < graph.offsets[u + 1]; i++) {

            int v = graph.neighbors[i];

            if (result.distance[v] == -1) {

                result.distance[v] = currentDistance + 1;

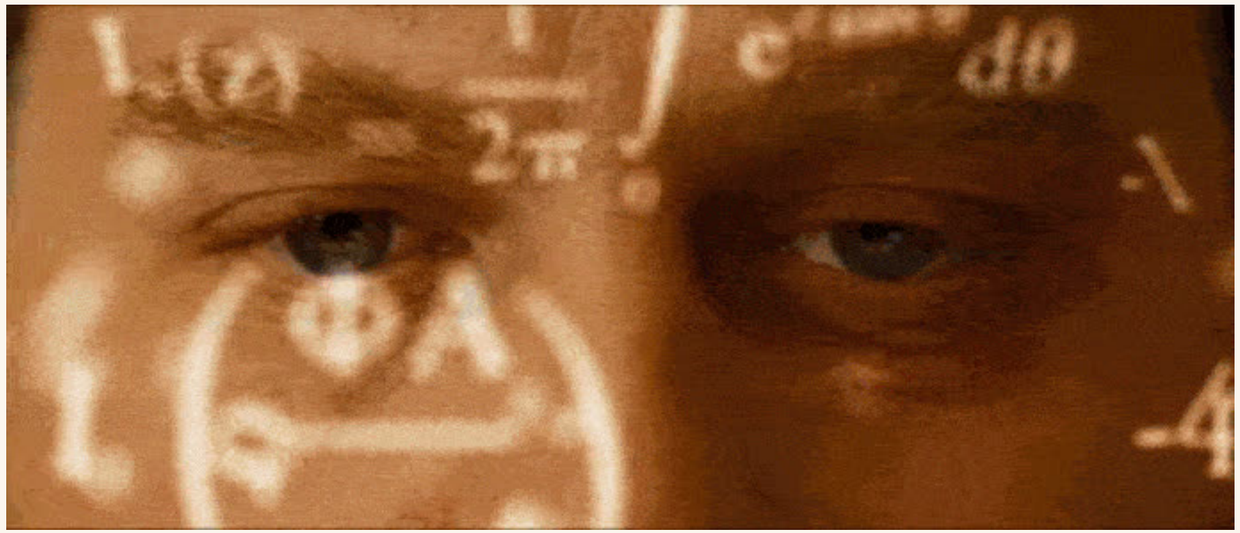
                frontier.push(v);
            }
        }
    }
    return result;
}
```

best possible sequential solution.

n = |V| คือจำนวน vertices
m = |E| คือจำนวน undirected edges

The algorithm takes $O(m+n)$

https://arxiv.org/pdf/1906.03113v2



Core Concepts

Frontier
Frontier คือกลุ่มของ Vertex ที่ค้นพบแล้วและรอการขยายไปยัง Neighbor

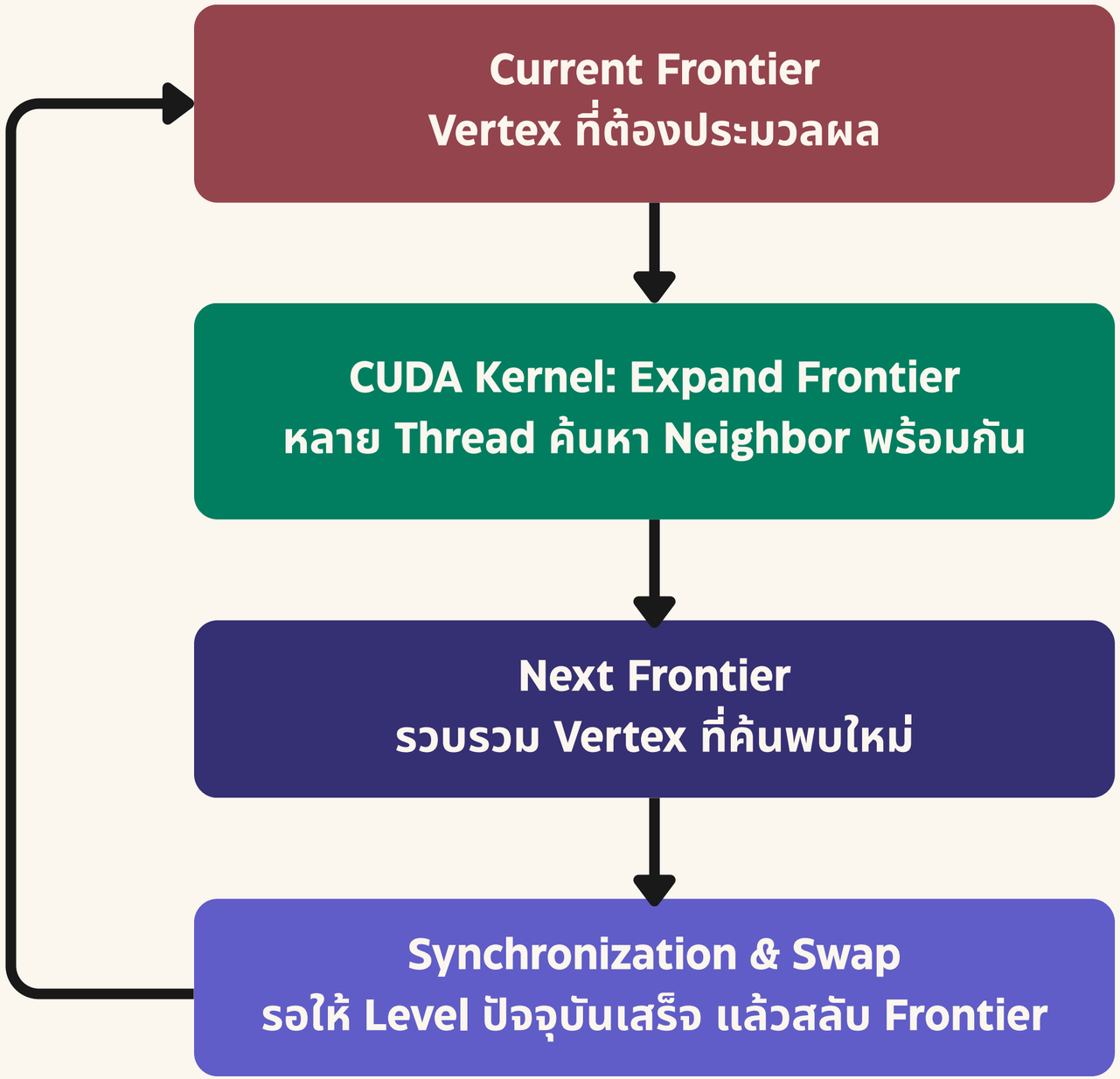
```
int* currentFrontier = d_frontierA;
int* nextFrontier = d_frontierB;
```

currentFrontier : เก็บ Vertex ที่ต้องประมวลผลใน Level ปัจจุบัน
nextFrontier : เก็บ Vertex ใหม่ที่ค้นพบจากการประมวลผล Level ปัจจุบัน

GPU จะนำ vertex ใน currentFrontier ไปประมวลผลพร้อมกัน เมื่อค้นพบ Neighbor ใหม่ ก็เพิ่มเข้า nextFrontier
เมื่อประมวลผลเสร็จ จะสลับ Frontier เพื่อเริ่ม Level ถัดไป

Core Concepts

Level-Synchronous
เราใช้การประมวลผลแบบ Level-Synchronous หมายถึง ต้องประมวลผล Frontier ของ Level ปัจจุบันให้เสร็จก่อน จึงจะเริ่ม Level ถัดไป



Approaches to Kernel

CUDA Kernel ของเรา มีการกำหนดให้หนึ่ง CUDA Thread รับผิดชอบหนึ่ง Vertex จาก Current Frontier

atomicCAS - compare in Atomic

```
int atomicCAS(int *address, int compare, int val)
```

```
// Claim vertex only once.
if (atomicCAS(&distance[v], -1, currentLevel + 1) == -1) {

    int position = atomicAdd(nextSize, 1);

    nextFrontier[position] = v;
}
```

why one would make senses

Brent's Theorem:
W = Work หรือจำนวนงานทั้งหมด
S/D = Depth หรือจำนวนขั้นตอนที่ต้องทำตามลำดับ
p = จำนวน processors

Work (W)
Initialize distance array: (O(n))
Visit each reachable vertex: (O(n))
Examine adjacency entries: (O(2m))
Queue operations: (O(n))
Ans : W = O(n+m)

Depth (D)
BFS ต้องค้นหาเป็นระดับ (Level-Synchronous) เพราะต้องค้นพบ vertices ในระดับก่อนหน้าให้เสร็จก่อน จึงจะเริ่มระดับถัดไปได้
Ans : D = O(n+m)



why one would make senses

Brent's Theorem:

$$T_p = O\left(\frac{W}{p} + D\right)$$

$$T_p = O\left(\frac{n+m}{p} + (n+m)\right)$$

กรณี p = 1

$$T_1 = O\left(\frac{n+m}{1} + (n+m)\right)$$

$$T_1 = O(n+m)$$

why one would make senses

Brent's Theorem:

The algorithm takes $O(m+n)$ < $T_p = O\left(\frac{n+m}{p} + (n+m)\right)$

Implementing the sequential solution in terms of CPU code

```h
#ifndef GRAPH_H
#define GRAPH_H

#include <vector>
#include <cstdint>

struct Graph {
    int numNodes = 0;
    int64_t numEdges = 0;

    std::vector<int> offsets;
    std::vector<int> neighbors;
    std::vector<int> originalIds;
};

struct BFSResult {
    std::vector<int> distance;

    int source = -1;
    int reachedVertices = 0;
    int maxDistance = 0;

    double executionTime = 0.0;
};

Graph loadGraph(const char* filename);

BFSResult runSequentialBFS(
    const Graph& graph,
    int source
);

BFSResult runParallelBFS(
    const Graph& graph,
    int source
);

#endif
```

```cpp
#include "graph.h"

#include <vector>
#include <queue>
#include <chrono>

BFSResult runSequentialBFS(
    const Graph& graph,
    int source
) {
    BFSResult result;
    result.source = source;

    result.distance.assign(graph.numNodes, -1);

    std::queue<int> frontier;

    auto start = std::chrono::high_resolution_clock::now();

    // Level 0
    result.distance[source] = 0;
    frontier.push(source);

    while (!frontier.empty()) {

        int u = frontier.front();
        frontier.pop();

        result.reachedVertices++;

        int currentDistance = result.distance[u];

        if (currentDistance > result.maxDistance) {
            result.maxDistance = currentDistance;
        }

        // Explore neighbors
        for (int i = graph.offsets[u];
             i < graph.offsets[u + 1]; i++) {

            int v = graph.neighbors[i];

            if (result.distance[v] == -1) {

                result.distance[v] = currentDistance + 1;

                frontier.push(v);
            }
        }
    }

    auto end = std::chrono::high_resolution_clock::now();

    result.executionTime =
        std::chrono::duration<double, std::milli>(
            end - start
        ).count();

    return result;
}
```

Implementing the parallel solution in terms of GPU code



```
__global__ void expandFrontier(
    const int* offsets,
    const int* neighbors,
    int* distance,
    const int* currentFrontier,
    int currentSize,
    int* nextFrontier,
    int* nextSize,
    int currentLevel
) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    if (tid >= currentSize)
        return;

    int u = currentFrontier[tid];

    int start = offsets[u];
    int end = offsets[u + 1];

    for (int i = start; i < end; i++) {

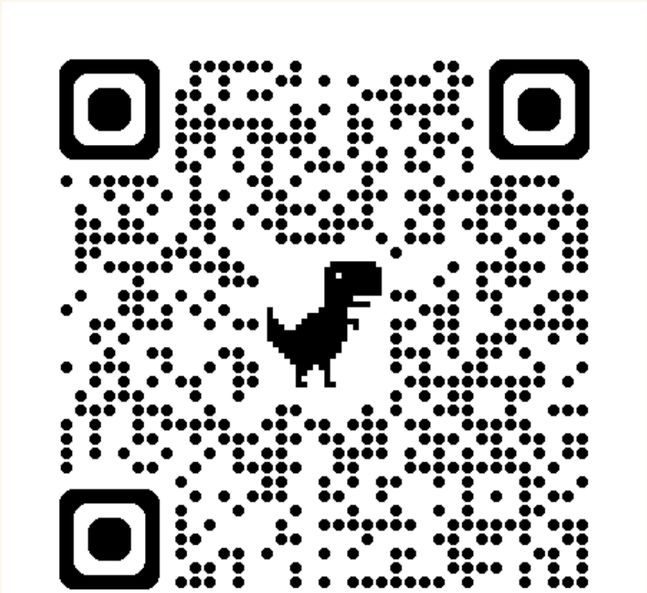
        int v = neighbors[i];

        // Claim vertex only once.
        if (atomicCAS(&distance[v], -1, currentLevel + 1) == -1) {

            int position = atomicAdd(nextSize, 1);

            nextFrontier[position] = v;
        }
    }
}
```

Thank you for your attention.



Feel free to check it out and share your feedback.

References

https://en.wikipedia.org/wiki/Graph_traversal

https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/

https://www.cs.cmu.edu/~guyb/paralg/paralg/parallel.pdf

https://arxiv.org/pdf/1906.03113v2

https://medium.com/@codeinseoul/introduction-to-graph-algorithms-a462ddc86783

https://research.nvidia.com/sites/default/files/pubs/2011-08_High-Performance-and/BFS%20TR.pdfhttps://en.wikipedia.org/wiki/Parallel_breadth-first_search